Slomins Notification History — Control4 Driver

Overview

Complete Control4 driver for viewing notification history from all Slomins devices with CldBus API integration, video playback, device filtering, and automatic refresh.

Version: 2
Package: NotificationHistory.c4z
Minimum Control4 OS: 3.3.2+
Created: June 8, 2026
Modified: June 16, 2026

Change History

Version 2 - June 16, 2026

- Fixed an issue where entering a random email could still show "Login Successful," but no devices or data were retrieved afterward.
- Fixed an issue where devices could temporarily disappear.

Purpose

This driver provides a centralized notification history viewer in Control4, allowing users to:

- View notification history from all Slomins devices (cameras, locks, doorbells, etc.)
- Play back video clips directly in the Control4 interface
- Filter notifications by specific devices
- Auto-refresh every 10 seconds with latest events

- MAC address validation for secure account access

Key Features

✅ Notification History

- View up to 20 most recent notifications across all devices
- Display device name, event type, timestamp, and thumbnail
- Automatic polling every 10 seconds for new events
- Formatted timestamps (e.g., "6/5/2026, 2:30 PM")

✅ Video Playback

- HTML5 video player with full controls
- Play/pause, seek, volume control
- AWS S3 pre-signed URLs (15-minute validity)
- Automatic URL refresh on polling

✅ Device Filtering

- Dropdown to filter notifications by device
- Shows device name and type (e.g., "Outdoor Camera(Solar)")
- "All Devices" option to view complete history
- Filter persists during automatic updates

✅ MAC Address Validation

- Validates Control4 MAC address against Slomins API
- Secures account access to authorized systems only
- Auto-sets MAC on driver installation
- Re-authenticates when MAC or email changes

✅ Modern UI

- Dark theme matching Control4 interface

- Responsive design for touch panels and mobile
- Thumbnail preview for each notification
- Video duration display

Key Files

- `driver.lua` — Main driver: initialization, authentication, API calls, device management, polling.
- `driver.xml` — Driver manifest, properties, and actions.
- `www/contents/index.html` — WebView UI with video player and device filtering.
- `CldBusApi/*` — Helper libraries:
  - `dkjson.lua` — JSON encoding/decoding
  - `http.lua` — HTTP request builder
  - `transport_c4.lua` — C4 network transport
  - `util.lua` — UUID generation, HMAC signatures
  - `auth.lua` — Authentication helpers

Driver Properties

Property	Type	Default	Description
MacAddress	STRING	Auto-set	Control4 system MAC address (validated on startup)
Shieldlink Account Email	STRING	Empty	User's Slomins account email (triggers re-auth on change)
Validation API URL	STRING	qa2.slomins.com/QA/.../IsValidControl4MacAddress	API endpoint for MAC validation

Property	Type	Default	Description
Base API URL	STRING	https://api.arpha-tech.com	CldBus API base URL
Account	STRING	cgabu@slomins.com	Account email for API authentication
ClientID	STRING	Auto-generated	UUID for this driver instance
Public Key	STRING	Empty	RSA public key from init API (for encryption)
Auth Token	STRING	Empty	Bearer token for API requests (secured)
AppId	STRING	cldbus	Application identifier
AppSecret	STRING	Secured	Application secret (password protected)
Status	STRING	Initializing...	Current driver status (read-only)

Initialization Flow

1. **OnDriverInit()**
   - Loads properties into `_props` table
   - Establishes TCP connection to tuyadev.slomins.net:8081
   - Schedules `InitializeCamera()` after 2 seconds
2. **OnDriverLateInit()**

- Sets `MacAddress` property using `C4:GetUniqueMAC()`
- Validates MAC address via API
- Clears device list if validation fails

3. **InitializeCamera()**
   - Generates new ClientID (UUID v4)
   - Creates HMAC-SHA256 signature
   - Calls `/api/v3/openapi/init` to retrieve public key
   - On success, proceeds to `LoginOrRegister()`

4. **LoginOrRegister()**
   - Encrypts account email using RSA-OAEP-SHA256 (external API)
   - Calls `/api/v3/openapi/auth/login-or-register`
   - Stores auth token in properties
   - Sends token to Node API for distribution

5. **TCP Receive (LnduUpdate Event)**
   - Receives encrypted message from TCP server
   - Decrypts using AES-256-CBC
   - Extracts Token, AppId, AppSecret
   - Updates properties and calls `UpdateAuthToken()`

6. **UpdateAuthToken()**
   - Stores auth token
   - Calls `GET_DEVICES()` to fetch device list

7. **GET_DEVICES()**
   - Fetches all devices from `/api/v3/openapi/devices-v2`
   - Builds `ALL_DEVICES` and `ALL_VIDS` arrays
   - Sends device list to UI via `SendDevicesToUI()`
   - Fetches notification history via `FETCH_NOTIFICATION_HISTORY()`
   - Starts 10-second polling timer

API Endpoints Used

Endpoint	Method	Purpose
/api/v3/openapi/init	POST	Retrieve RSA public key for encryption
/api/v3/openapi/auth/login-or-register	POST	Authenticate user and get auth token
/api/v3/openapi/devices-v2	GET	List all devices for the account
/api/v3/openapi/notifications/query	POST	Fetch notification history (20 items, paginated)
http://54.90.205.243:5000/lndu-encrypt	POST	External RSA-OAEP encryption service
http://54.90.205.243:3000/send-to-control4	POST	Send token to Node API for distribution

Notification Data Structure

Each notification contains:

```
{
  device_name: "Outdoor Camera(Solar)",
  vid: "ebfb1d2c3a4b5c6d7e8f",
  time: 1717612800,  // Unix timestamp
  image_url: "https://s3.amazonaws.com/...",
  video_url: "https://s3.amazonaws.com/...",
  video_sec: 15,  // Duration in seconds
  message_type: "motion_detected"
}
```

UI Communication Pattern

The driver uses Control4's WebView communication pattern discovered from Smart-Lock driver:

Driver → UI (Data Push)

```
-- Send notification history
local payload = { type = "history", history = LAST_HISTORY }
local jsonPayload = C4:JsonEncode(payload)
C4:SendToProxy(5001, "ICON_CHANGED", { icon_description = jsonPayload })
C4:SendToProxy(5001, "UPDATE_UI", {})

-- Send device list
local payload = { type = "device_list", devices = LAST_DEVICES }
local jsonPayload = C4:JsonEncode(payload)
C4:SendToProxy(5001, "ICON_CHANGED", { icon_description = jsonPayload })
C4:SendToProxy(5001, "UPDATE_UI", {})
```

UI → Driver (Subscription)

```
// Subscribe to driver data updates
C4.subscribeToDataToUi(false);

// Receive data from driver
function onDataToUi(value) {
  var parsed = JSON.parse(value);
  var data = JSON.parse(parsed.icon_description);

  if (data.type === 'history') {
    // Handle notification history
    ALL_NOTIFICATIONS = convertToArray(data.history);
    applyFilter();
  }

  if (data.type === 'device_list') {
    // Handle device list
    DEVICES = convertToArray(data.devices);
    populateDeviceFilter();
  }
}
```

Lua JSON Quirk (Object-to-Array)

Lua's `table.insert()` creates 1-indexed tables, which `dkjson` encodes as objects:

```
-- Lua output
{"1": {...}, "2": {...}, "3": {...}}
```

```
-- JavaScript expects
[{...}, {...}, {...}]
```

Solution: JavaScript converts objects to arrays:

```
function convertToArray(obj) {
  if (Array.isArray(obj)) return obj;
  var arr = [];
  for (var key in obj) {
    if (obj.hasOwnProperty(key)) {
      arr.push(obj[key]);
    }
  }
  return arr;
}
```

Device Filtering

The UI maintains two notification arrays:

- `ALL_NOTIFICATIONS` — Unfiltered list (all 20 notifications)
- `NOTIFICATIONS` — Filtered subset based on device selection

Filter Logic

```
function applyFilter() {
  if (!selectedDeviceVid || selectedDeviceVid === '') {
    // Show all devices
    NOTIFICATIONS = ALL_NOTIFICATIONS;
  } else {
    // Filter by VID
    NOTIFICATIONS = ALL_NOTIFICATIONS.filter(function(n) {
      return n.vid === selectedDeviceVid;
    });
  }
  renderList();
}
```

Filter Persistence

When new data arrives every 10 seconds, the filter re-applies automatically:

```
if (data.type === 'history') {
  ALL_NOTIFICATIONS = convertToArray(data.history);
  applyFilter();  // Maintains current filter
}
```

Property Change Handlers

MacAddress Changed

```
if strName == "MacAddress" then
  C4:UpdateProperty("MacAddress", Properties[strName])
  ClearDeviceList()
  ValidateMacAddress(Properties["MacAddress"], function(isValid)
    if isValid then
      InitializeCamera()
    end
  end)
end
```

Shieldlink Account Email Changed

```
if strName == "Shieldlink Account Email" then
  local email = Properties["Shieldlink Account Email"]
  ClearDeviceList()

  if email == "" then
    C4:UpdateProperty("Status", "Enter email")
    return
  end

  ValidateMacAddress(Properties["MacAddress"], function(isValid)
    if isValid then
      C4:UpdateProperty("Account", email)
      InitializeCamera()
    end
  end)
end
```

Actions

Action	Command	Description
Test: Fetch Clips	TEST_FETCH_CLIPS	Manually fetch notification history (debug/testing)

Troubleshooting

No Notifications Showing

Check:

1. MAC address is validated (**Status** property shows "MAC address validated")
2. Account email is correct in **Account** or **Shieldlink Account Email**
3. **Auth Token** property is populated
4. At least one device exists in your account
5. Network can reach `api.arpha-tech.com`

Action:

1. Check Composer logs for: `📹 VIDEO CLIPS FOUND: X`
2. Run **Test: Fetch Clips** action
3. Verify **Total devices fetched: X** in logs
4. Check if devices have recent motion events

MAC Validation Failed

Check:

1. **MacAddress** property contains valid MAC
2. Validation API URL is accessible
3. MAC is registered in Slomins system
4. Network firewall allows HTTPS to qa2.slomins.com

Action:

1. Verify MAC address in Composer properties
2. Check logs for validation error messages
3. Contact Slomins support to register MAC
4. Try manual MAC entry if auto-detection fails

Device Filter Not Populating

Check:

1. **Auth Token** is valid
2. `GET_DEVICES()` was called (check logs)
3. Devices exist in account
4. WebView received device list data

Action:

1. Reload the notification history UI
2. Check logs for: `Total devices fetched: X`
3. Verify: `✅ Sent device list to UI via ICON_CHANGED`
4. Check browser console for JavaScript errors

Video Won't Play

Check:

1. Video URL is valid (15-minute S3 signed URL)
2. URL hasn't expired (check timestamp)
3. Network can reach AWS S3
4. Video file exists for that notification

Action:

1. Wait for next polling cycle (10 seconds) to refresh URLs
2. Check if thumbnail loads (image_url)
3. Test video URL directly in browser
4. Verify notification has `video_url` field

Polling Stopped

Check:

1. Driver didn't crash (check logs for errors)
2. Auth token is still valid
3. Network connection to API is stable
4. `POLL_TIMER` is running

Action:

1. Reload driver in Composer
2. Check logs for: `Polling notifications...` every 10 seconds
3. Verify no API errors in logs
4. Re-initialize if auth token expired

Authentication Failures

Check:

1. **Account** email is valid Slomins account
2. **Public Key** was retrieved from init API
3. Encryption API (port 5000) is accessible
4. Network can reach external encryption service

Action:

1. Check logs for: `Camera initialized successfully`
2. Verify: `Public key received: [KEY]`
3. Check: `Login succeeded` in logs
4. Verify **Auth Token** property is filled

Data Not Updating

Check:

1. Polling is active (10-second timer)
2. No `Poll skipped (in-flight)` messages

3. API responses are successful (code 200)
4. WebView is subscribed to data updates

Action:

1. Close and reopen notification history UI
2. Check logs for data transmission: `Sending X notifications to UI`
3. Verify: ✅ `Sent notification data to UI via UPDATE_UI`
4. Reload driver if polling stopped

Technical Notes

Video URL Expiration

AWS S3 pre-signed URLs are valid for 15 minutes only. The driver refreshes URLs automatically:

- Polling every 10 seconds fetches new notification data
- Each poll regenerates S3 signed URLs
- UI updates automatically with fresh URLs
- Video remains playable continuously due to frequent updates

Polling Strategy

```
-- 10-second timer with in-flight protection
local function PollTick()
  if _pollingInFlight then
    print("Poll skipped (in-flight)")
  else
    _pollingInFlight = true
    FETCH_NOTIFICATION_HISTORY(vids, function()
      _pollingInFlight = false
    end)
  end
  POLL_TIMER = C4:SetTimer(POLL_INTERVAL, PollTick)
end
```

TCP Configuration

- **Server:** tuyadev.slomins.net
- **Port:** 8081
- **Protocol:** TCP with AES-256-CBC encryption
- **Purpose:** Receive auth tokens and app secrets from central server
- **Delimiter:** `0d0a` (CRLF)

AES Decryption

```
local decrypted = C4:Decrypt(
  "AES-256-CBC",
  "DMb9vJT7ZuhQsI967YUuV621SqGwg1jG",  -- 32-byte key
  "33rj6KNVN4kFvd0s",                   -- 16-byte IV
  data,
  {
    return_encoding = "NONE",
    key_encoding = "NONE",
    iv_encoding = "NONE",
    data_encoding = "BASE64",
    padding = true
  }
)
```

HMAC Signature Generation

```
local message = string.format(
  "client_id=%s&request_id=%s&time=%s&version=%s",
  client_id, request_id, time, version
)
local signature = util.hmac_sha256_hex(message, app_secret)
```

Developer Notes

Adding New Notification Types

To display new event types, update the UI message type mapping:

```
// In index.html
function getEventTypeLabel(type) {
  var types = {
    'motion_detected': 'Motion Detected',
    'human_detected': 'Person Detected',
    'doorbell': 'Doorbell Pressed',
    'lock_opened': 'Door Unlocked',
    // Add new types here
  };
  return types[type] || type;
}
```

Adjusting Polling Interval

```
-- In driver.lua (line ~237)
local POLL_INTERVAL = 10000  -- Change to 5000 for 5 seconds, 30000 for 30 seconds
```

Increasing Notification Count

```
-- In FETCH_NOTIFICATION_HISTORY() (line ~314)
local body = {
  page = 1,
  page_size = 20,  -- Change to 50 for more notifications
  vids = vids
}
```

Debugging Tips

- Check Composer Lua Output for detailed logs
- Look for: `📹 VIDEO CLIPS FOUND: X` to verify API responses
- Monitor: `✅ Sent notification data to UI` for data transmission
- Use browser Developer Tools (F12) for WebView JavaScript debugging
- Check `onDataToUi` console logs for received data
